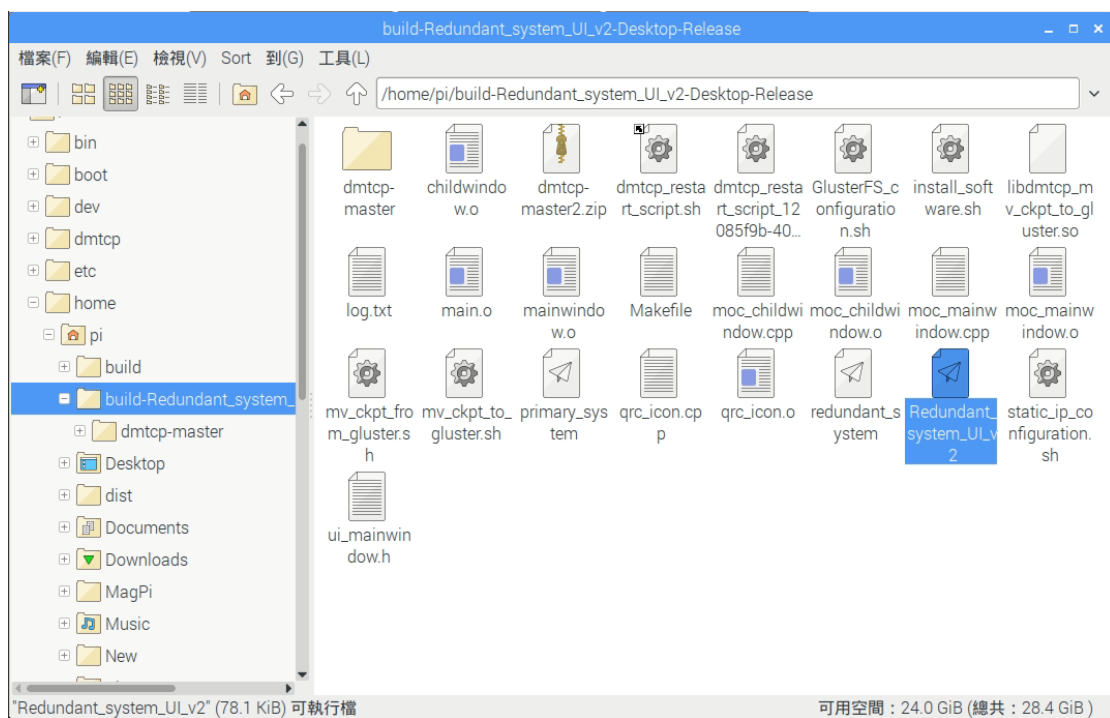


Redundant system 建置步驟，因為已經有 assistant tool 可以協助安裝，這邊就說明 assistant tool 的操作步驟。

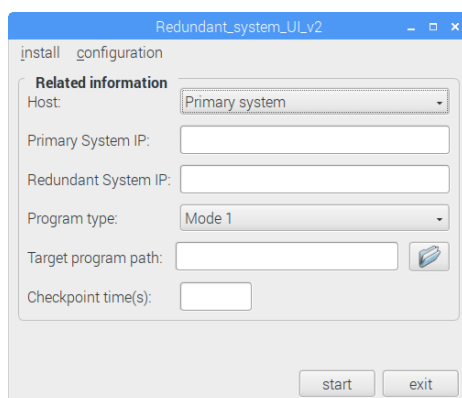
首先準備好兩台 raspberry pi 和兩張 SD 卡，在 SD 卡上安裝 raspbian 作業系統 <https://www.raspberrypi.org/downloads/raspbian/>，我用的版本是 2018-11-13-raspbian-stretch，系統安裝好後，將 build-Redundant_system_.....這個壓縮檔解壓縮，如下圖所示。

```
pi@raspberrypi: ~  
檔案(F) 編輯(E) 分頁(T) 說明(H)  
pi@raspberrypi:~ $ ls -l  
總計 38584  
drwxr-xr-x 3 pi pi 4096 8月 3 2018 build  
drwxr-xr-x 3 pi pi 4096 6月 14 12:55 build-Redundant_system_UI_v2-Desktop-Release  
-rw-r--r-- 1 pi pi 25087392 6月 13 13:37 build-Redundant_system_UI_v2-Desktop-Release.zip
```

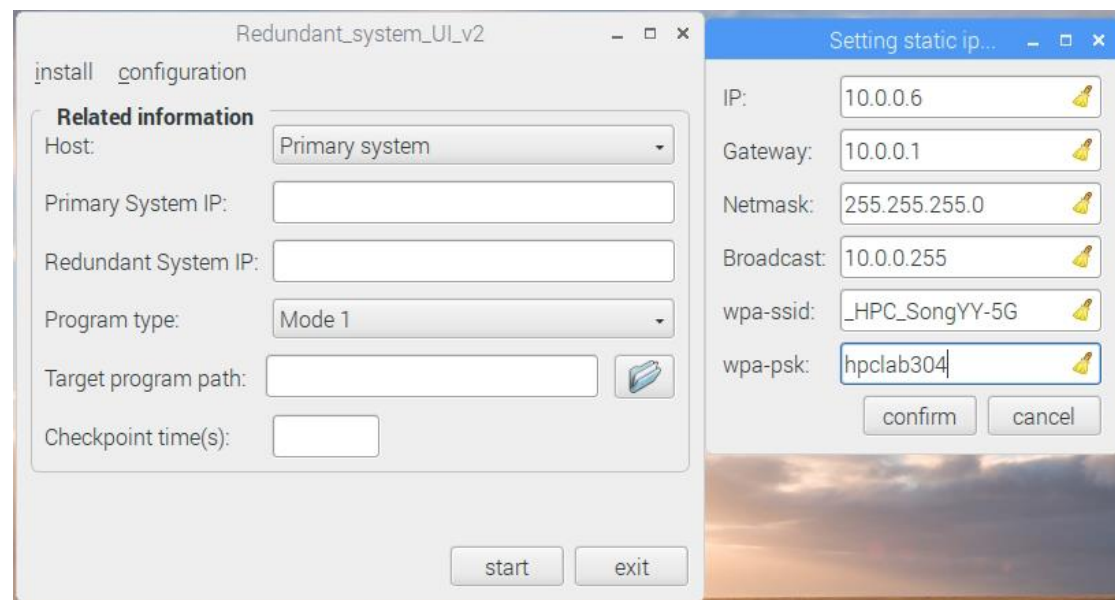
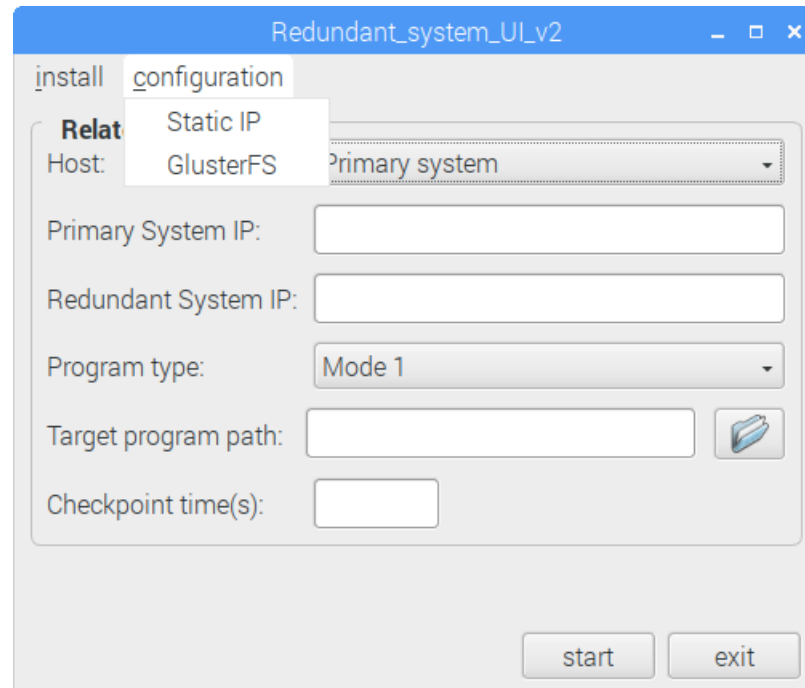
之後進入資料夾點 Redundant_system_UI_v2 後跳出對話框選執行

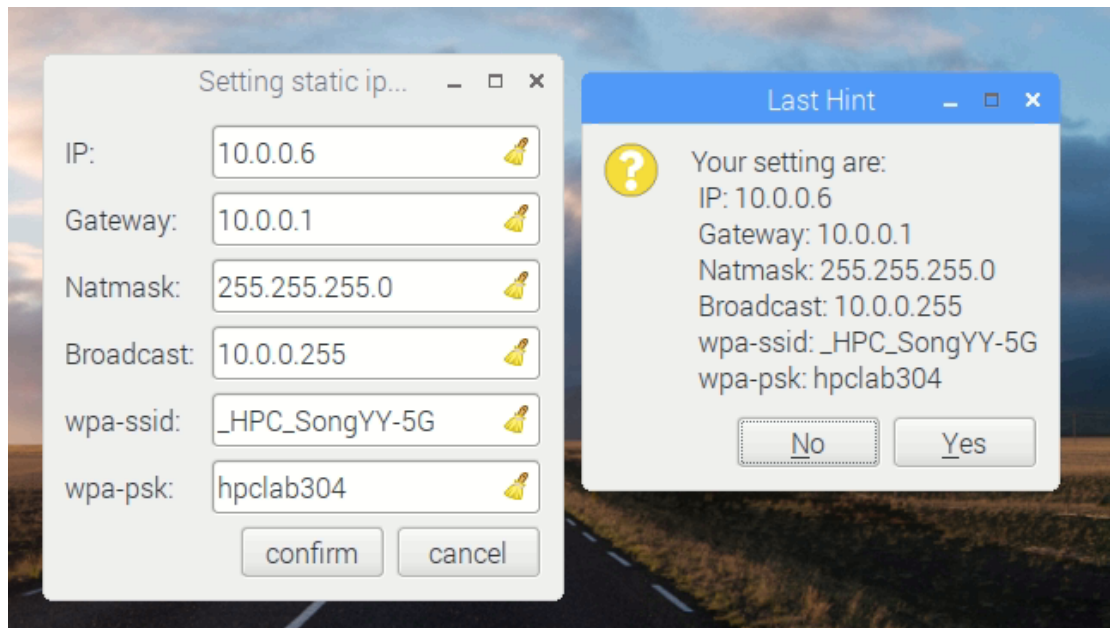


就會跳出這個畫面

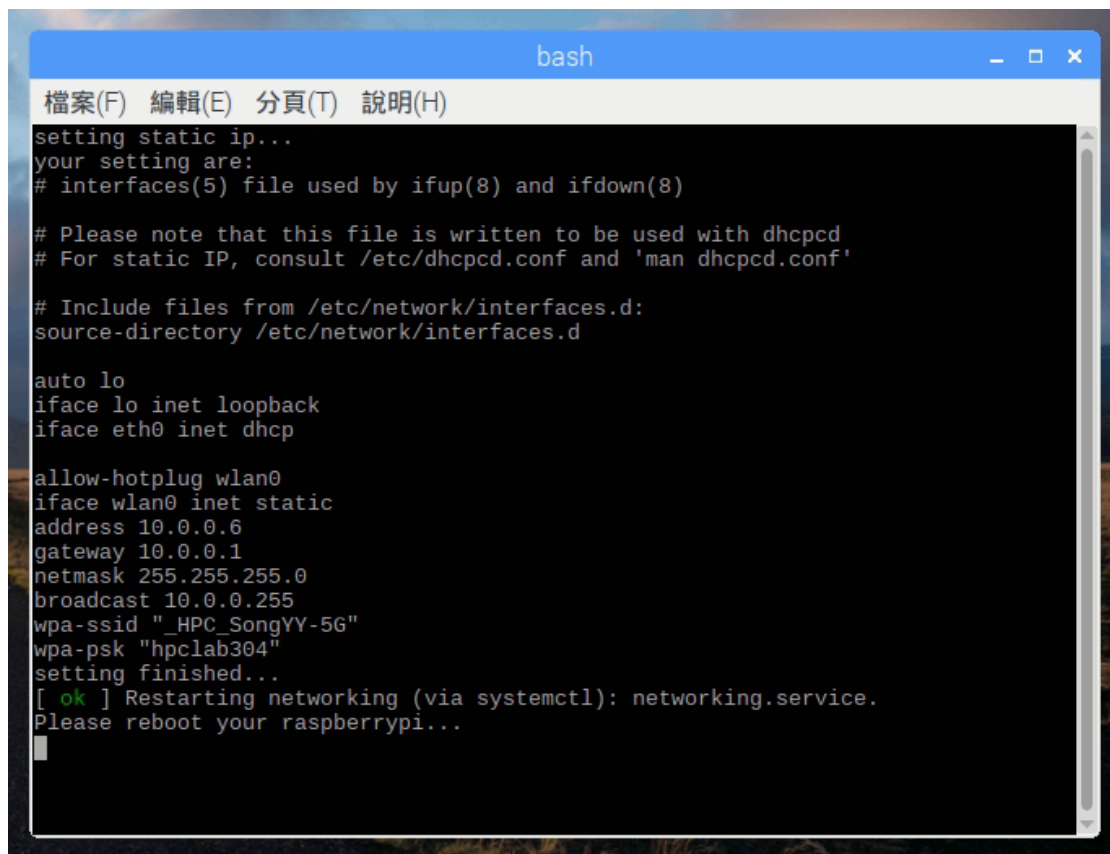


這裡就可以開始安裝需要用到的一些相關軟體，先點 **install** 分別去安裝 **glusterFS**、**DMTCP**、**Watchdog**，安裝 **DMTCP** 時會需要花一些時間。
安裝好後點 **configuration** 設定環境，先設定 **static ip** 再設定 **GlusterFS**。
PS: 這裡我連的是 **wifi**
如下圖所示





設定好後記得重新開機，設定的參數才會生效。



重新開機完後，可以檢查看看 IP 是否有更改成你設定的 IP。

```
pi@raspberrypi:~ $ ifconfig
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 3 bytes 157 (157.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 3 bytes 157 (157.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

wlan0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.0.6 netmask 255.255.255.0 broadcast 10.0.0.255
    inet6 fe80::ba27:ebff:fe73:3e78 prefixlen 64 scopeid 0x20<link>
    inet6 2002:8c7b:613e:0:ba27:ebff:fe73:3e78 prefixlen 64 scopeid 0x0<global>
    ether b8:27:eb:73:3e:78 txqueuelen 1000 (Ethernet)
    RX packets 470 bytes 37637 (36.7 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 834 bytes 919050 (897.5 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

pi@raspberrypi:~ $
```

※兩台 raspberry pi 都要安裝好軟體和設定好 static ip。

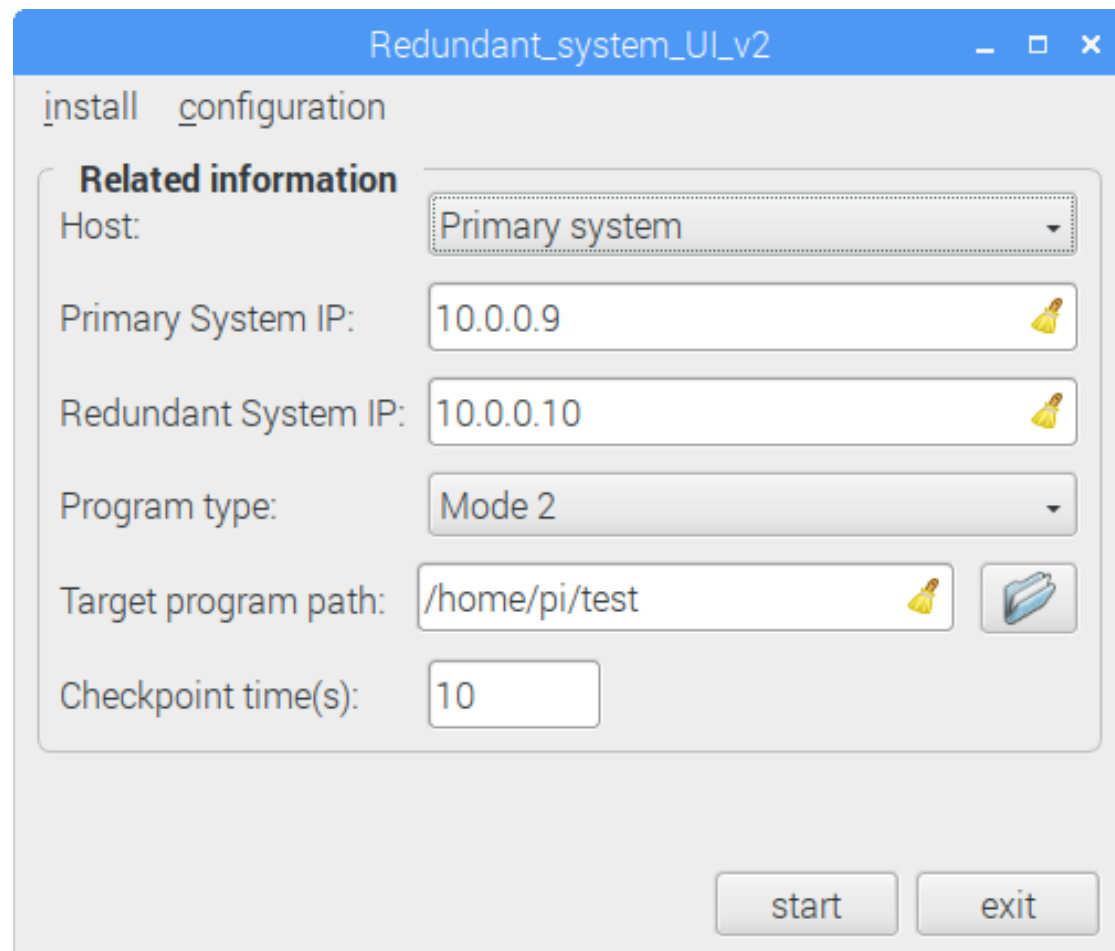
再來設定 GlusterFS，選好哪台 raspberry pi 要作為 primary system，將 Host 欄位選 Primary system，並將 primary system IP 和 redundant system IP 輸入欄位後點 configuration->GlusterFS，設定好後再換到另一台 raspberry pi 將 Host 欄位選 Redundant system，並將 primary system IP 和 redundant system IP 輸入欄位後點 configuration->GlusterFS。這裡要注意的是要先 primary system 設定好後再去設定 redundant system 否則會出問題。

Gluster 安裝好後可以下 `sudo gluster volume info` 指令，會出現相關資訊確認是否安裝好 gluster。

```
pi@raspberrypi-active:/ $ sudo gluster volume info

Volume Name: gv0
Type: Replicate
Volume ID: 168eae8d-0674-413b-8320-30e97c23ce05
Status: Started
Snapshot Count: 0
Number of Bricks: 1 x 2 = 2
Transport-type: tcp
Bricks:
Brick1: 10.0.0.17:/srv/gv0
Brick2: 10.0.0.18:/srv/gv0
Options Reconfigured:
transport.address-family: inet
performance.readdir-ahead: on
nfs.disable: on
pi@raspberrypi-active:/ $
```

再來依據程式的類型，在 Program type 的欄位選擇 Mode-1 或是 Mode-2，Mode-1 如 sensor 量測溫溼度程式; Mode-2 一般程式需要保存記憶體狀態的程式以便故障後恢復。最後 Target program path 輸入要執行程式的路徑。而 checkpoint time 欄位則是每幾秒執行一次 checkpoint。如圖所示(primary system)。



The screenshot shows a window titled "Redundant_system_UI_v2" with a blue header bar. Below the header, there are two tabs: "install" and "configuration", with "configuration" being the active tab. The main area is titled "Related information" and contains several input fields and buttons:

- Host:** A dropdown menu showing "Primary system".
- Primary System IP:** A text input field containing "10.0.0.9".
- Redundant System IP:** A text input field containing "10.0.0.10".
- Program type:** A dropdown menu showing "Mode 2".
- Target program path:** A text input field containing "/home/pi/test", with a folder icon button to its right.
- Checkpoint time(s):** A text input field containing "10".

At the bottom right of the window, there are two buttons: "start" and "exit".

而作為 redundant system，需要填寫的資訊和 primary system 差不多。Host 部分選擇 redundant system。比較不一樣的部分在 redundant system 上，如果要執行 Mode-1 的程式類型，需要將 target program 也放一份在 redundant system 上，這樣當 primary system crash 後，redundant system 才能執行 target program。而 Mode-2 程式類型，可以不用填寫 target program 的路徑和 checkpoint time. 因為 DMTCp 在幫我們 create checkpoint image 時就把原始設定的參數包在裡面了，因此直接 restart checkpoint image 即可。如下圖所示。

Redundant_system_UI_v2

install configuration

Related information

Host: Redundant system

Primary System IP: 10.0.0.9

Redundant System IP: 10.0.0.10

Program type: Mode 1

Target program path: /home/pi/dist/simpletest

Checkpoint time(s):

start exit

Redundant_system_UI_v2

install configuration

Related information

Host: Redundant system

Primary System IP: 10.0.0.9

Redundant System IP: 10.0.0.10

Program type: Mode 2

Target program path:

Checkpoint time(s):

start exit

最後點擊 start 就能開始用了，這裡無論是哪台 raspberry pi 先點 start 都不影響程式的執行。

Mode-1 的測試程式用 dht11_example.py 和 simpletest.py，也可以兩個都用 simpletest.py，因為我手上只有一個 Adafruit Python MAX31855 sensor 所以用一個 dht11_example.py (DHT11.zip\DHT11_Python\dht11_example.py) 另一個用 simpletest.py (test program 解壓縮後 dist 資料夾裡的 simpletest.py)。

Adafruit Python MAX31855 sensor 接法和安裝請參考勝文學長中的資料夾的南茂計畫南茂 Rpi-Setup。

Dht11.py 接法和安裝請參考: <https://www.youtube.com/watch?v=KUr8WgSIsfk>

Mode-2 測試程式用 test，test program 解壓縮後外面的 test。

如果要重新執行測試程式要把這兩個路徑/dmtcp 和 /mnt/gluster1 下的檔案清空，才不會出現問題。

安裝步驟我有錄影，請參考 hpcloud 我的資料夾，assistant tool 下的影片資料夾。

1.以上是思凱學姊的運行以及安裝，當你使用熟悉之後可以使用我的修改版本，我的 github 網址是 https://github.com/pengsheng-chen/SLR/tree/main/build-Redundant_system_UI_v2-Desktop-Release，請將 SLR/primary system/final_chage_socket/ 以及 SLR/redundant system/final_chage_socket/底下的 3 個檔案放入/build-Redundant_system_UI_v2-Desktop-Release/，取代原本的 primary_system 和 redundant_system，裡面也有原始碼可以修改。接下來操作在影片中都有。DEMO 影片網址:<https://youtu.be/iLYaJf2KeLY>

2.如果需要測量記憶體極限，可以參考 SLR/sys_limit/，裡面也有說明書可以教你如何做測量。DEMO 影片網址:<https://youtu.be/iLYaJf2KeLY>

3.接下來程式是運行容錯服務以及記憶體的監測程式，一樣方式，將 SLR/SLR_memory tracking/ 全部檔案放入/home/pi 底下，可以進行測試，在 build-Redundant_system_UI_v2-Desktop-Release 裡面的 top.sh 是在做抓取極限值。top.sh 裡面路徑需要自行修改” limitemem=\$(sed -n '2p' /home/pi/final.txtlawk '{print \$1}')minimum=\$(sed -n '1p' /home/pi/final.txtlawk '{print \$1}'))”，修改成自己的檔案位置。詳細測試可以參考，DEMO 影片網址:<https://youtu.be/iLYaJf2KeLY>。

4.最後你可能是修改 mtrace 的論文，請先進入 SLR/mtrace_install/將 glibc.sh 放入你的 home 底下執行，就可以幫你完成編譯一個全新的 glibc 使用，請注意修改 glibc 版成自己的版本，我這裡使用 glibc_2.31。接下來 glibc_install.txt 可以幫助你指定自己的 glibc 編譯程式，詳細情形請查看我的論文。等你使用 mtrace 追蹤，會生成日誌檔，在使用 mtrace_analysis.sh 做 memort_leak 和 double free 分析。如果以上沒問題可以參考編譯我的 glibc_2.31，也在 github 中有。這個版本可以幫助你完成論文中的功能。

5.由於我論文中的 vovol 的論文利用的 benchmark 做測試，發現在 mibench 中的 dijkstra_large 會導致運行時間大增，詳細時間可以參考 SLR/Mtrace_speed up/裡面的 PPT，同時我也做了相關更改，可以降低執行時間。但是不是每個 benchmark 都會導致執行時間大增，例如: mibench 中的 FFT 運行時間不會相差太多。所以 SLR/Mtrace_speed up/裡面的 mtrace 可以取代 glibc_2.31 原本的 mtrace，可以幫助你加速。